

PLATAFORMA WEB TRANSACCIONAL PARA LA GESTIÓN DE ALTOS DE CARPAY

GESTIÓN DE ARRIENDOS, MOTOR DE RESERVAS Y CONTROL DE PAGOS

NOMBRE: Enrique Alegría, Vicente Martin, Leonardo Rivera y Fernanda Salgado
CARRERA: Ingeniería en informática / TI3043
ASIGNATURA: Ingeniería de software
PROFESOR: Manuel Antonio Reveco Cabello
FECHA: 20/10/2026



Índice

PLATAFORMA WEB TRANSACCIONAL PARA LA GESTIÓN DE ALTOS DE CARPAY 1

1 Introducción 3

2 Identificación del Proyecto 4

2.1 Resumen de la Problemática: 4

2.2 Objetivo General: 4

2.3 Objetivos Específicos: 5

3 Metodología de Desarrollo de Software 5

3.1 Justificación de la metodología seleccionada: Enfoque hibrido (Scrum + Desarrollo incremental) 5

4 Técnicas de cocreación y levantamiento de requerimientos utilizados 6

4.1 Product Backlog 8

4.2 Sprint Backlog 9

4.2.1 Sprint 1: Estructura Base, Catálogo y Gestión de Usuarios 9

4.2.2 Sprint 2: Motor de Reservas, Pagos y Notificaciones 10

5 Diagramas de la problemática 13

6 Elección del patrón de diseño 16

Diagrama de Clases según el Patrón de Diseño (MVC) 17

7 Estándares de calidad 19

7.1 Selección de Normas 19

7.2 Justificación de su Elección y Aplicación al Proyecto 19

7.2.1 Aplicación del Estándar ISO/IEC 25010 19

7.2.2 Aplicación del Estándar WCAG 2.1 (Nivel AA) 20

8 Conclusión 21

9 Glosario 22

10 Bibliografía 23



1 Introducción

El presente informe detalla el desarrollo de la segunda etapa del proyecto web “Altos de Carpay”, iniciativa orientada a dar solución a la problemática de difusión, gestión de reservas y atención de clientes de un emprendimiento familiar de arriendo de terrenos para eventos. Para lograr una implementación estructurada y eficiente, el trabajo se ha basado en la selección de una metodología híbrida que combina el Desarrollo Incremental para la evolución del software y Scrum para la organización del equipo de trabajo. Asimismo, se documenta el levantamiento formal de requerimientos mediante historias de usuario organizadas en un Product Backlog y Sprint Backlog, la modelación de la problemática a través de diagramas de comportamiento y estructurales, la definición del patrón de diseño arquitectónico y la incorporación de estándares internacionales de calidad de software que garantizan la fiabilidad, mantenibilidad y accesibilidad de la plataforma digital.



2 Identificación del Proyecto

2.1 Resumen de la Problemática:

El emprendimiento familiar "Altos de Carpay", dedicado al arriendo de terrenos para eventos, enfrenta una serie de limitaciones operativas e informativas debido a la carencia de un canal digital formal de difusión. Actualmente, la promoción de las instalaciones, la consulta de precios, la verificación de disponibilidad y el proceso de atención a clientes se realizan de forma totalmente manual, dependiendo de contactos verbales y mensajería directa.

Esta modalidad genera fricción y lentitud en la atención, ya que obliga a los administradores a responder de forma reiterativa las mismas consultas básicas sobre capacidad, ubicación y servicios. Asimismo, la falta de un sistema automatizado para el control de fechas y registro de operaciones incrementa el riesgo de traspaso de información incorrecta, descoordinación en el cobro de anticipos y demoras en el cierre de reservas, limitando el potencial de crecimiento y el alcance comercial del negocio.

2.2 Objetivo General:

Diseñar e implementar una plataforma web interactiva y accesible para la gestión integral de arriendos y difusión de "Altos de Carpay", estructurada mediante el patrón arquitectónico Modelo-Vista-Controlador (MVC con Django y React), que automatice el proceso de consulta de disponibilidad, el cálculo de anticipos correspondientes al 30% del valor total y la caducidad de reservas con 24 horas de antelación, bajo estándares internacionales de calidad y accesibilidad de software.



2.3 Objetivos Específicos:

1) Diseñar un catálogo público e interactivo: Desarrollar una interfaz de usuario limpia, intuitiva y accesible bajo las pautas WCAG 2.1, que exhiba la oferta de terrenos con sus respectivas fotografías, ubicaciones, capacidades, tarifas y condiciones de uso.

2) Implementar un módulo de autenticación y gestión de roles: Configurar un control de acceso seguro que diferencie las funcionalidades y permisos entre los usuarios en rol de Cliente y los en rol de Administrador.

3) Habilitar el motor de reservas y gestión de pagos: Desarrollar el sistema de selección de fechas/horarios en tiempo real y la pasarela para el cobro del 30% de anticipo o el pago total, integrando la regla de negocio que verifica el saldo restante hasta 24 horas antes del evento.

4) Integrar un servicio automatizado de notificaciones: Configurar el envío automático de correos electrónicos de confirmación con el desglose detallado del arriendo, montos abonados y saldos pendientes tras concretarse una reserva o pago.

3 Metodología de Desarrollo de Software

3.1 Justificación de la metodología seleccionada: Enfoque hibrido (Scrum + Desarrollo incremental)

Para llevar a cabo el desarrollo del proyecto web "Altos de Carpay", el equipo ha optado por implementar una estrategia híbrida que combina el Desarrollo Incremental en el ámbito del producto de software y el marco de trabajo Scrum en la gestión del equipo y la relación con el cliente.

- **Ámbito del Producto (Metodología Incremental):** Permite estructurar la construcción del sitio web en bloques funcionales y evolutivos. El sistema se desarrolla por incrementos progresivos (desde la estructura base informativa y catálogo, hasta el motor de reservas, pasarelas de pago y panel administrativo), asegurando que cada versión cumpla con la estabilidad necesaria antes de incorporar la siguiente fase.



- **Ámbito de la Gestión del Equipo y Cliente (Metodología Scrum):** Permite organizar las tareas de los cuatro integrantes del grupo mediante ciclos iterativos o Sprints, priorizar los requerimientos mediante un Product Backlog, asignar responsabilidades claras en un Sprint Backlog y mantener una retroalimentación constante con el representante del emprendimiento al finalizar cada entrega.

4 Técnicas de cocreación y levantamiento de requerimientos utilizados

Para el levantamiento de los requerimientos y la conceptualización de la página web informativa de Altos de Carpay, se emplearon técnicas basadas en la colaboración directa con el representante del negocio, permitiendo modelar tanto la problemática de difusión como las bases funcionales del sitio web.



Técnicas Utilizadas	Descripción y propósito	Sitio web
Entrevistas Semi-estructuradas con el Stakeholder	Reuniones directas y guiadas con el representante del emprendimiento (Lucas Súi Ziebold Máximo) para indagar en los procesos actuales, las dificultades de difusión y las necesidades de los clientes.	Permitió identificar que la difusión dependía de contactos verbales y comunicaciones manuales, delimitando el problema central y estableciendo los requerimientos de la página informativa (fotografías, ubicación, capacidad, servicios y condiciones de uso).
Modelado de Procesos del Negocio Actual y Propuesto	Técnica de análisis visual y descriptivo que contrasta la situación actual del negocio frente al flujo mejorado tras la incorporación del sistema web.	Facilitó trazar el nuevo flujo: 1.Potencial cliente → 2.Ingreso al sitio web → 3.Consulta de información y servicios → 4.Evaluación → 5.Contacto con el administrador, reduciendo la



		fricción en la entrega de información básica.
Talleres de Definición de Alcance y Escalabilidad	Sesiones de trabajo en conjunto para determinar los límites funcionales del software, separando el módulo web informativo de los procesos complejos de control de ingreso/salida.	Ayudó a definir formalmente las limitaciones del proyecto actual y a estructurar una visión escalable para futuras etapas (incorporando paulatinamente registros, inicio de sesión, calendarios de disponibilidad y sistema de reviews).

4.1 Product Backlog

El Product Backlog agrupa y prioriza las historias de usuario necesarias para el cumplimiento de los objetivos del sistema:

ID	Historia de Usuario	Prioridad
HU-01	Como visitante, quiero revisar los terrenos y sus características para elegir el espacio adecuado.	Alta
HU-02	Como cliente, quiero registrarme e iniciar sesión para gestionar mis reservas.	Alta
HU-03	Como cliente, quiero comprobar fechas y horarios disponibles para solicitar una reserva.	Alta



HU-04	Como cliente, quiero conocer el precio detallado antes de pagar.	Media
HU-05	Como cliente, quiero pagar el 30% de anticipo o el total para reservar el terreno.	Alta
HU-06	Como cliente, quiero recibir un correo con los detalles de mi operación.	
HU-07	Como administrador, quiero mantener el catálogo y los precios para que la información esté actualizada.	Alta
HU-08	Como administrador, quiero revisar y gestionar reservas y consultas para mantener el control.	Media
HU-09	Como cliente, quiero conocer el saldo y la fecha límite de pago (hasta 24 horas antes) para evitar que mi reserva venza.	Media

4.2 Sprint Backlog

A continuación, se detalla la planificación de los ciclos de trabajo (Sprints) con los cinco componentes obligatorios:

4.2.1 Sprint 1: Estructura Base, Catálogo y Gestión de Usuarios

- Objetivo del Sprint: Implementar la interfaz principal del sitio web, la navegación del catálogo de terrenos y el sistema de registro/autenticación para clientes y administradores.
- Duración: 2 semanas.



Historia de usuario	Criterio de aceptación	Responsable	Rol
HU-01: Como visitante, quiero revisar los terrenos y sus características para elegir el espacio adecuado.	Dado que ingreso al sitio web, cuando abro el catálogo público, entonces puedo consultar los terrenos disponibles, fotografías, capacidad y tarifas.	Vicente Martin	Equipo de Desarrollo
HU-02: Como cliente, quiero registrarme e iniciar sesión para gestionar mis reservas.	Dado que ingreso datos válidos, cuando completo el registro o inicio sesión, entonces el sistema procesa la autenticación y aplica los permisos de mi rol.	Enrique Alegría	Product Owner / Encargado de BD
HU-07: Como administrador, quiero mantener el catálogo y los precios para que la información esté actualizada.	Dado que inicio sesión como administrador, cuando modifico o agrego un terreno, entonces el catálogo público refleja los cambios inmediatamente.	Fernanda Salgado	Scrum Master

4.2.2 Sprint 2: Motor de Reservas, Pagos y Notificaciones

- Objetivo del Sprint: Habilitar el módulo de selección de fechas/horarios en el calendario, la opción de pago de anticipo (30%) o total, y la emisión automatizada de correos de confirmación.
- Duración: 2 semanas.



Historia de usuario	Criterio de aceptación	Responsable	Rol
HU-03: Como cliente, quiero comprobar fechas y horarios disponibles para solicitar una reserva.	Dado que selecciono un terreno del catálogo, cuando consulto el calendario, entonces el sistema muestra únicamente los días y bloques horarios disponibles.	Leonardo Rivera	Equipo de Desarrollo
HU-05: Como cliente, quiero pagar el 30% de anticipo o el total para reservar el terreno.	Dado que se calcula el precio total, cuando elijo la opción de pago (30% o 100%), entonces el sistema procesa la transacción, registra la reserva y calcula el saldo pendiente.	Leonardo Rivera	Equipo de Desarrollo
HU-06: Como cliente, quiero recibir un correo con los detalles de mi operación.	Dado que se confirma el pago o reserva, cuando el sistema procesa la transacción, entonces me envía un correo electrónico con el desglose del arriendo, pago realizado y saldo pendiente.	Vicente Martin	Equipo de Desarrollo
HU-09: Como cliente, quiero conocer el saldo y	Dado que pagué solo el anticipo, cuando reviso el detalle de mi reserva,	Enrique Alegría	Product Owner / Encargado de BD



la fecha límite para evitar que mi reserva venza	entonces el sistema me muestra el saldo restante y me recuerda que debe saldarse hasta 24 horas antes del evento.		



5 Diagramas de la problemática

5.1 Diagrama de caso de uso

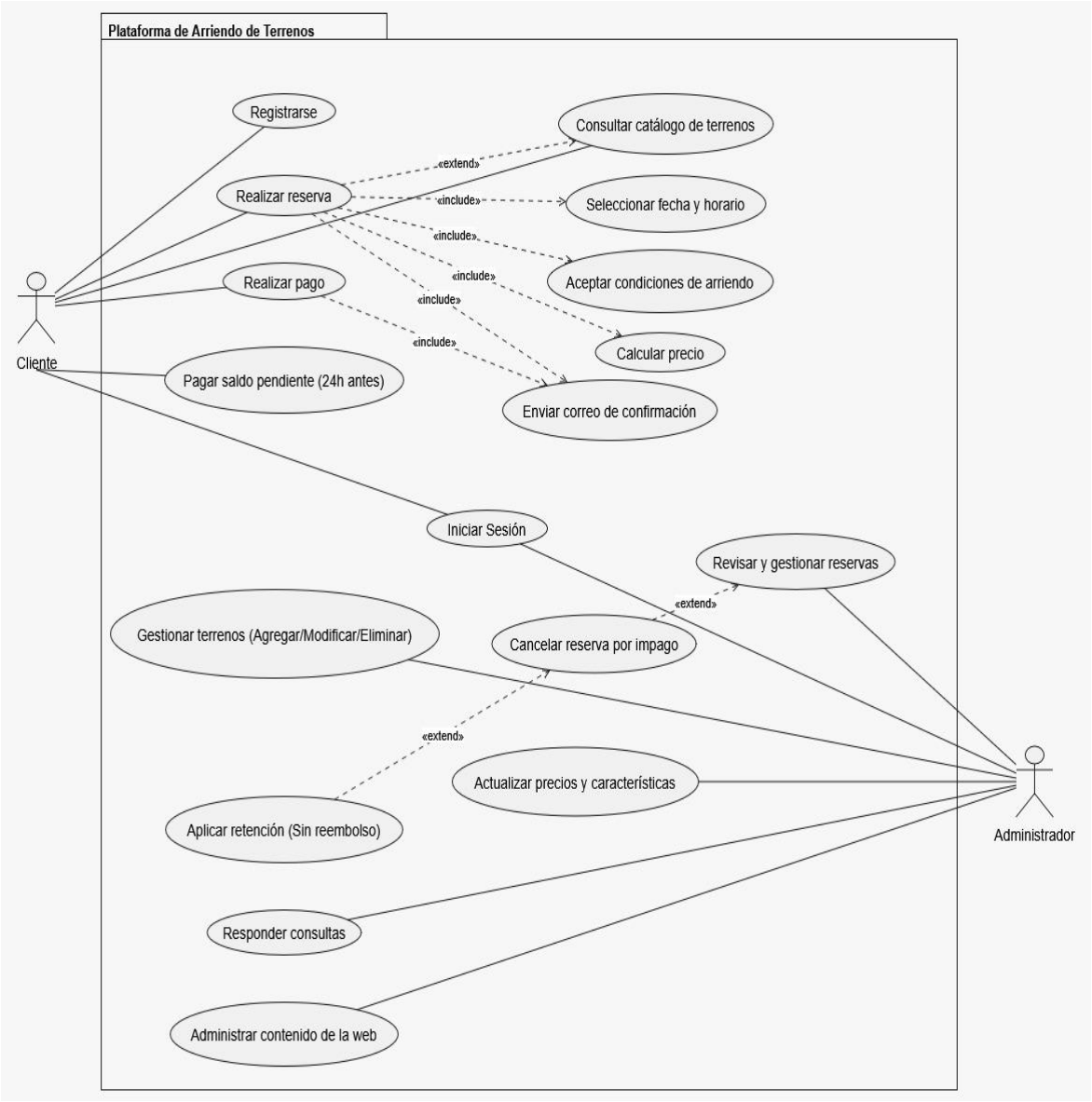


Ilustración 1 Diagrama de Casos de Usos



5.2 Diagrama de secuencia

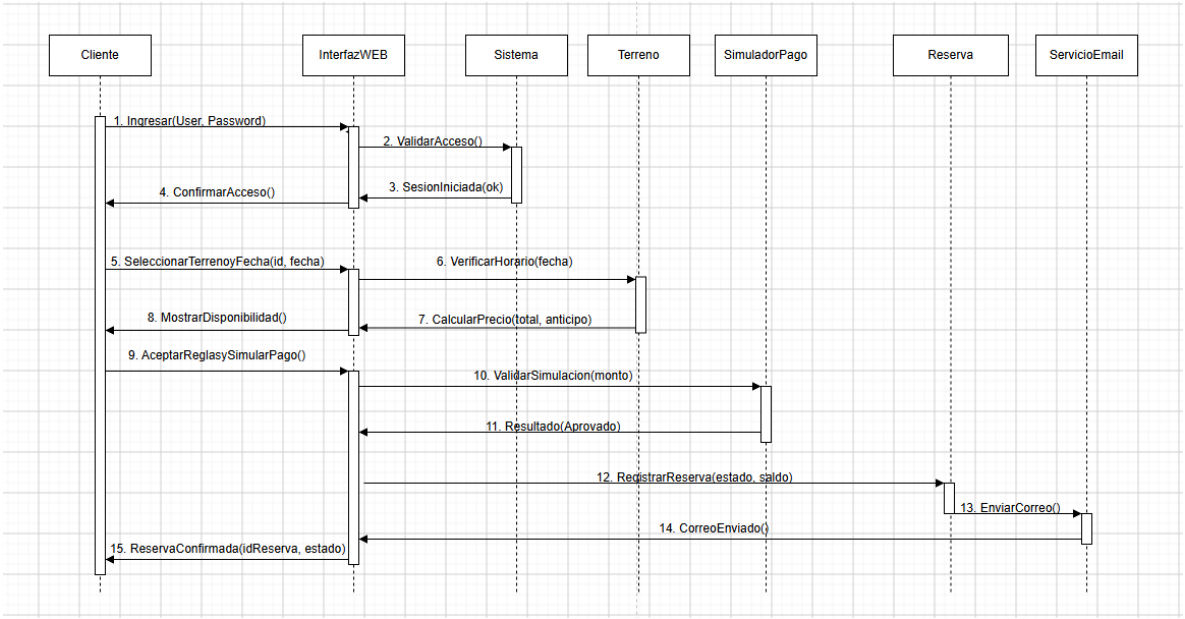


Ilustración 2 Diagrama de Secuencias



5.3 Diagrama de clase

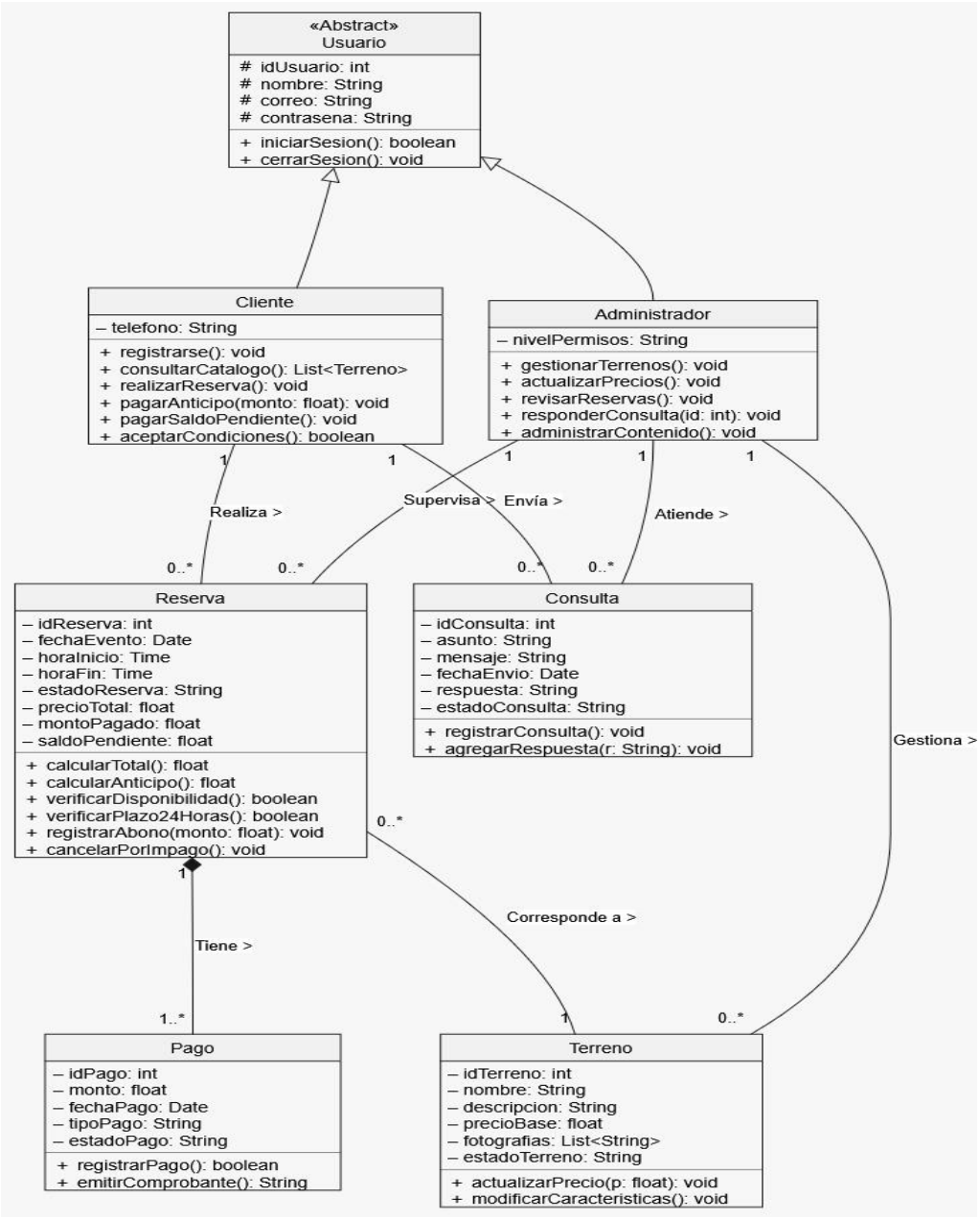


Ilustración 3 Diagrama de Clases



6 Elección del patrón de diseño

Para dar solución a la problemática del sistema web de arriendo de terrenos en Altos de Carpay, se ha seleccionado el patrón arquitectónico Modelo-Vista-Controlador (MVC), implementado de manera desacoplada a través del framework Django (Backend / API REST) en el servidor y React (Frontend) en la capa de presentación del cliente.

La elección de este patrón y su distribución tecnológica se justifica técnicamente por:

1. **Separación Estricta de Responsabilidades:** Permite aislar la interfaz de usuario (desarrollada en React y alojada en un servidor web Nginx) de la lógica de negocio central (módulos de reservas, cálculo de anticipos del 30% y el planificador automatizado de cancelaciones por impago a las 24 horas). La comunicación entre ambas capas se realiza exclusivamente mediante endpoints de una API REST protegida por autenticación y control de tokens.
    1) **Capa Modelo (Django ORM / PostgreSQL):** Administra los datos, la persistencia y las reglas de negocio del sistema. Modela entidades principales como Usuario, Cliente, Administrador, Terreno, Reserva y Pago, garantizando transacciones seguras (como la reserva con el 30% de anticipo y la regla de vencimiento a las 24 horas).
    2) **Capa Vista (React / Frontend):** Encargada del despliegue de la interfaz gráfica e interacción con el usuario. Renders del catálogo interactivo, formularios de registro, selección de fechas en el calendario y paneles de control.
    3) **Capa Controlador (Django REST Framework / API):** Actúa como intermediario entre la Vista y el Modelo. Recibe las solicitudes HTTP desde React, valida los datos de entrada, ejecuta las reglas de negocio sobre el Modelo y retorna respuestas estructuradas en formato JSON.
2. **Persistencia y Coherencia de Datos:** El modelo de dominio del sistema (Usuario, Cliente, Administrador, Terreno, Reserva, Pago y Consulta) interactúa directamente con una base de



datos relacional PostgreSQL mediante el ORM de Django y su driver correspondiente, asegurando la integridad transaccional en cada operación de arriendo.

3. **Escalabilidad y Mantenibilidad:** La estructura modular facilita la distribución de tareas entre los integrantes del equipo de desarrollo, permitiendo realizar pruebas unitarias, despliegues contenerizados y futuras ampliaciones del sistema de forma limpia y ordenada.
   El desacoplamiento permite realizar modificaciones en la interfaz visual (React) sin afectar la lógica de negocio ni la estructura de la base de datos (PostgreSQL).
   Facilita la incorporación futura de nuevas plataformas (como una aplicación móvil) reutilizando los mismos controladores y modelos expuestos vía API REST.
4. **Seguridad y Control de Transacciones:** La validación de permisos, autenticación mediante tokens y control de roles (Cliente vs. Administrador) se ejecutan de manera centralizada en la capa de controladores en el servidor, evitando vulnerabilidades en el cliente.

El diseño del diagrama de clase contempla una clase abstracta base denominada Usuario, la cual se especializada en los roles de Cliente y Administrador. A partir de estos actores, el modelo gestiona las entidades clave del negocio: los Terrenos disponibles para eventos, las Reservas asociadas a fechas y horarios específicos, los registros de Pago correspondientes a anticipos o saldos pendientes, y el módulo de Consultas para la atención de usuarios.

Diagrama de Clases según el Patrón de Diseño (MVC)

El siguiente diagrama representa la estructura estática del sistema organizada y segregada de acuerdo con las tres capas del patrón MVC:



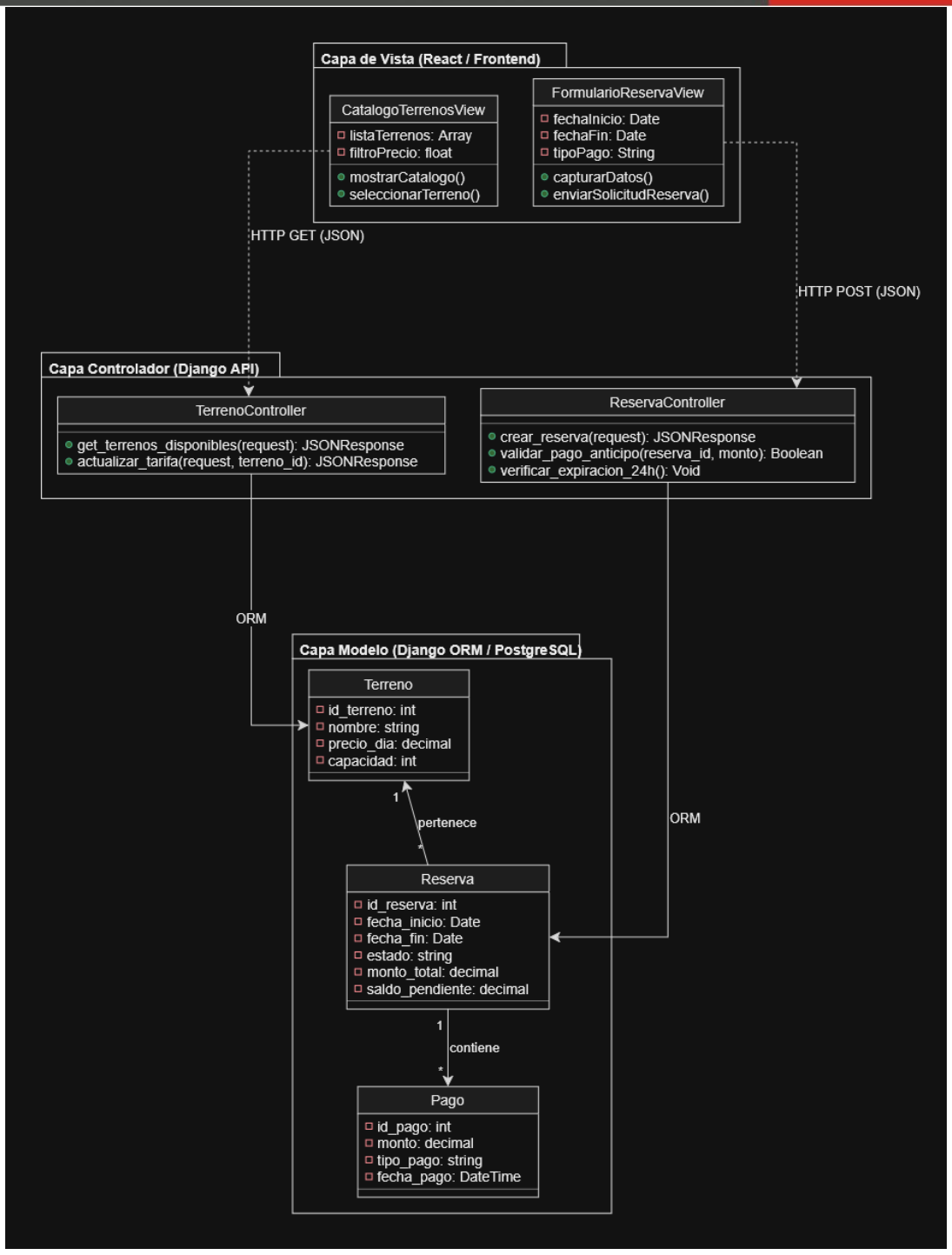


Ilustración 4 Diagrama de Clases Adaptado al Patrón de Diseño (Modelo-Vista-Controlador / MVC)



7 Estándares de calidad

Para asegurar que la plataforma web de "Altos de Carpay" cumpla con altos niveles de exigencia técnica, seguridad transaccional, mantenibilidad y accesibilidad universal, se han seleccionado dos normativas internacionales reconocidas en la industria del software:

7.1 Selección de Normas

ISO/IEC 25010 (System and Software Quality Models): Norma internacional que establece un modelo de calidad del producto de software estructurado en características de calidad para evaluar sistemas computacionales.

WCAG 2.1 (Web Content Accessibility Guidelines - Nivel AA): Pautas internacionales de accesibilidad web emitidas por el W3C (World Wide Web Consortium) orientadas a garantizar que el contenido digital sea utilizable por la mayor cantidad de personas, incluyendo aquellas con necesidades especiales o capacidades diferentes.

7.2 Justificación de su Elección y Aplicación al Proyecto

7.2.1 Aplicación del Estándar ISO/IEC 25010

Dado que el sistema "Altos de Carpay" administra operaciones financieras (cálculo de montos, cobro del 30% de anticipo y control de saldos) junto con reglas de tiempo críticas (como la caducidad automática de solicitudes sin pago a las 24 horas), se han priorizado cuatro pilares fundamentales del modelo de calidad:

1) **Adecuación Funcional:** La plataforma garantiza que las funciones de consulta de disponibilidad, gestión del catálogo de terrenos, registro de usuarios y procesamiento de pagos satisfagan plenamente las necesidades operativas del emprendimiento y los clientes.
2) **Fiabilidad (Tolerancia a fallos e Integridad):** La arquitectura basada en Django REST Framework e interacción transaccional con la base de datos PostgreSQL previene inconsistencias de datos (como la duplicidad de reservas en una misma fecha) y asegura la correcta recepción de pagos e información sensible de los usuarios.
3) **Mantenibilidad:** La aplicación del patrón de diseño arquitectónico Modelo-Vista-Controlador (MVC), sumada a la separación física entre el Frontend en React y el Backend en Django, asegura un código modular, documentado y fácil de actualizar o corregir en el tiempo.
4) **Eficiencia de Desempeño:** Promueve un tiempo de respuesta optimizado durante la consulta de disponibilidad en el calendario e interacción con las imágenes del catálogo, asegurando un consumo eficiente de recursos del servidor.



7.2.2 Aplicación del Estándar WCAG 2.1 (Nivel AA)

La adopción de las pautas WCAG 2.1 responde a la necesidad de ofrecer una plataforma inclusiva para el público general que desea cotizar y reservar terrenos para eventos. Se implementan las directrices basándose en sus cuatro principios rectores:

1) **Perceptible:** Las imágenes del catálogo de terrenos incluyen alternativas textuales en sus atributos (alt), y se utilizan esquemas de color con contraste adecuado entre el texto y el fondo para facilitar la lectura de tarifas y condiciones de uso.

2) **Operable:** La navegación por la interfaz, la exploración del catálogo y la selección de fechas en el formulario de reserva están optimizadas para ser ejecutadas tanto mediante el puntero del mouse como a través de la navegación secuencial por teclado.

3) **Comprensible:** Los formularios de registro, inicio de sesión y solicitud de reserva cuentan con etiquetas claras, mensajes de validación instructivos e indicaciones explícitas paso a paso para evitar errores en el ingreso de datos.

4) **Robusto:** El marcado HTML5 y la estructuración de componentes en React garantizan la compatibilidad con múltiples navegadores (*Chrome, Firefox, Safari, Edge*) y dispositivos móviles o de escritorio.



8 Conclusión

En conclusión, el desarrollo de la plataforma web para “Altos de Carpay” representa una respuesta tecnológica sólida frente a la carencia de un canal formal de difusión y gestión en el emprendimiento. La aplicación rigurosa de la metodología de desarrollo incremental y el marco de trabajo Scrum permitió organizar al equipo de cuatro integrantes en ciclos iterativos bien definidos, facilitando la trazabilidad de los requerimientos desde el levantamiento de información hasta el diseño de la arquitectura. La elección del patrón Modelo-Vista-Controlador (MVC) con Django y React, sumada al cumplimiento de los estándares ISO/IEC 25010 y WCAG 2.1, asegura que el sistema no solo sea funcional, seguro y coherente en el manejo transaccional de reservas y pagos, sino también escalable y accesible para cualquier usuario, sentando bases firmes para la digitalización y el crecimiento futuro del negocio.



9 Glosario

1. **API REST (Application Programming Interface - Representational State Transfer):** Interfaz de programación de aplicaciones que utiliza solicitudes HTTP para obtener, enviar y manipular datos en formato estructurado (generalmente JSON) entre un servidor y un cliente web o móvil.
2. **Django:** Framework de desarrollo web de alto nivel y código abierto escrito en Python, utilizado para construir el Backend y la API REST del sistema mediante el patrón Modelo-Vista-Controlador (MVC/MVT).
3. **Gherkin:** Lenguaje de especificación de requerimientos orientado al negocio que utiliza una sintaxis estructurada (*Dado que / Cuando / Entonces*) para definir criterios de aceptación probables e historias de usuario en metodologías ágiles.
4. **ISO/IEC 25010:** Norma internacional dictada por la Organización Internacional de Normalización que establece el modelo de calidad para productos de software, evaluando características como adecuación funcional, fiabilidad, mantenibilidad y eficiencia de desempeño.
5. **MVC (Modelo-Vista-Controlador):** Patrón de diseño arquitectónico que divide una aplicación en tres componentes interconectados: el Modelo (datos y reglas de negocio), la Vista (interfaz de usuario) y el Controlador (gestor de peticiones y flujo de datos).
6. **Product Backlog:** Lista priorizada y dinámica que contiene todas las historias de usuario, funcionalidades, requerimientos y tareas necesarias para la construcción completa del producto de software.
7. **React:** Librería JavaScript de código abierto desarrollada por Meta, enfocada en la creación de interfaces de usuario interactivas, modulares y reactivas en la capa del Frontend.
8. **Sprint Backlog:** Subconjunto de elementos seleccionados del Product Backlog que el equipo de desarrollo se compromete a completar durante la duración de un ciclo de trabajo o Sprint determinado.
9. **WCAG 2.1 (Web Content Accessibility Guidelines):** Conjunto de recomendaciones y pautas desarrolladas por el W3C (*World Wide Web Consortium*) dictadas para hacer que el contenido web sea accesible para el mayor número de personas, priorizando los principios de perceptibilidad, operabilidad, comprensibilidad y robustez.



10 Bibliografía

1) **ISO/IEC. (2011).** Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models (ISO/IEC Standard No. 25010:2011). International Organization for Standardization. https://www.iso.org/standard/35733.html
2) **Pressman, R. S., & Maxim, B. R. (2021).** Ingeniería del software: Un enfoque práctico (9.ª ed.). McGraw-Hill Education.
3) **W3C. (2018).** Web Content Accessibility Guidelines (WCAG) 2.1 (W3C Recommendation). World Wide Web Consortium. https://www.w3.org/TR/WCAG21/



inacap

APRENDER HACIENDO A LO LARGO DE LA VIDA

ACREDITADO

N

7AÑOS

MÁXIMA

EXCELENCIA

CNA

Comisión Nacional de Acreditación Chile

INSTITUTO PROFESIONAL INACAP

ACREDITADO • NIVEL DE EXCELENCIA

- Gestión Institucional • Docencia de Pregrado
- Vinculación con el Medio
- Próxima Acreditación Septiembre 2030

www.inacap.cl